

1. What should you include in a checkpoint to resume training seamlessly? 1 point
- ☐ {"model_state_dict", "optimizer_state_dict", "learning_rate", "batch_size"}
 - ☒ {"model_state_dict", "optimizer_state_dict", "epoch", "loss"}
 - ☐ {"model_state_dict", "epoch", "loss", "best_accuracy"}
 - ☐ {"optimizer_state_dict", "epoch", "loss", "random_seed"}
2. Why do you need to explicitly save trained models in PyTorch? 1 point
- ☐ PyTorch automatically saves models to disk, but only the final version after training completes
 - ☒ Trained models only exist in RAM
 - ☐ To compress the model files so they take up less disk space
 - ☐ Notebooks automatically save all variables, but not PyTorch model objects
3. Why does torch.onnx.export() require an example input tensor? 1 point
- ☐ To validate that the model produces correct outputs before exporting
 - ☒ ONNX needs to know the shape of the input your model expects
 - ☐ To include sample data inside the ONNX file for testing purposes
 - ☐ To set the fixed batch size that the ONNX model will use at inference time
4. In torch.onnx.export(), what does do_constant_folding=True do? 1 point
- ☐ Converts all weights to int8 to reduce model size
 - ☐ Enables variable-length inputs at runtime
 - ☒ Precomputes constant calculations and stores the results
 - ☐ Forces the model to use the latest ONNX opset version
5. Which statement about structured vs. unstructured pruning is correct? 1 point
- ☐ Unstructured pruning always shrinks files and speeds up inference dramatically
 - ☐ Structured pruning is typically less harmful to accuracy than unstructured pruning
 - ☒ Structured pruning removes whole neurons/filters
 - ☐ Both structured and unstructured pruning physically remove weights from the model automatically in PyTorch
6. In prune.In_structured() with dim=0 and amount=0.5, what gets pruned from the layer? 1 point
- ☐ 50% of the inputs, selected randomly
 - ☒ 50% of the outputs, selected by importance
 - ☐ 50% of the inputs, selected by importance
 - ☐ 50% of the outputs, selected randomly
7. Why is 32-bit floating-point precision important during training but not necessarily required for inference? 1 point
- ☐ Training requires GPU computation which only supports 32-bit, while inference can run on CPUs with lower precision
 - ☒ Training accumulates small gradient updates where precision loss compounds, but fixed trained weights tolerate rounding
 - ☐ 32-bit precision prevents overfitting during training, while lower precision helps generalization during inference
 - ☐ Training data contains more variation requiring higher precision, while inference data is more uniform
8. When should you use dynamic quantization versus static quantization? 1 point
- ☐ Dynamic for CNNs where convolutions need quantization, static for RNNs and transformers with large linear layers
 - ☒ Dynamic for RNNs and transformers with large linear layers, static for CNNs where convolutions need quantization
 - ☐ Dynamic when you have calibration data available, static when you don't
 - ☐ Static in all cases since it produces smaller, faster models with better accuracy
9. During quantization-aware training (QAT), what does a "fake quantization" module do to activations and weights? 1 point
- ☐ Permanently converts them to int8 so the model trains with actual quantized values
 - ☐ Stores both float32 and int8 versions side-by-side throughout training
 - ☒ Quantizes the value to int8 then immediately dequantizes it back to float32
 - ☐ Skips quantization during training and only applies it at inference time
10. Why should you call fuse_modules() before prepare_qat() on sequences like convolution-batchnorm-ReLU? 1 point
- ☐ Fusing reduces memory usage by combining layer weights into a single tensor
 - ☐ PyTorch requires layers to be fused before any quantization can be applied
 - ☒ Fusing lets quantization see the full transformation and set appropriate output ranges
 - ☐ Fusing prevents the model from learning incorrect quantization scales during training

